HR Tech: The "Smart Talent" Selection Engine

1. Background

The Talent Acquisition department is currently bottlenecked by "Volume Hiring" cycles. When a single job posting attracts 1,000+ applicants, recruiters rely on primitive keyword-matching tools (ATS) to filter talent. This creates two critical points of failure:

First, **keyword-based filtering is "dumb."** If a job description asks for a "Java Developer" and a highly qualified candidate writes "Backend Specialist with JVM expertise," the system may reject them because the exact word was missing. This leads to the accidental rejection of high-potential talent who use synonymous terminology or creative formatting.

Second, the sheer volume leads to **"Manual Fatigue."** Recruiters spend an average of only 6 seconds per resume. In this rush, they often miss deep technical experience or project-based achievements, relying instead on "Keyword Stuffing" by candidates who have learned to game the system.

This creates a high-stakes problem for the company: we are missing the best talent while spending hundreds of man-hours manually reviewing resumes that shouldn't have made it past the first round. We need a system that understands **meaning** rather than just matching characters.

---

3. Detailed Feature Requirements

Feature 1: Multi-Format Resume Ingestion Portal

Build a secure upload portal where recruiters can bulk-upload resumes. The system must be robust enough to handle the chaotic variety of modern resume designs.

- **Requirements:**
    - Support for PDF, DocX, and high-quality image formats (JPG/PNG).
    - **Intelligent Parsing:** The system must handle non-linear layouts, such as two-column resumes, sidebars, and tables, without merging unrelated text blocks.
    - **Validation:** Show upload progress and provide feedback if a file is corrupt or in an unsupported format.
    - **Organization:** Group uploaded resumes by "Job Role" or "Batch Date" for easy management.

Feature 2: Semantic Profile & Intent Mapper

Build a logic engine that moves beyond keyword matching. This feature should transform unstructured resume text into a standardized technical profile based on the **intent** of the experience.

- **Requirements:**
    - **Skill Mapping:** Recognize synonyms and hierarchies (e.g., understand that "PyTorch" is a subset of "Machine Learning" or "React" relates to "Frontend").
    - **Experience Extraction:** Automatically distinguish between "Professional Experience," "Academic Projects," and "Certifications."
    - **Profile Normalization:** Convert extracted data into a structured JSON-like format that can be easily compared against other candidates.

Feature 3: JD-to-Candidate Ranking Dashboard

Build a feature that allows a recruiter to upload a specific Job Description (JD) and see a logically ranked list of candidates from the uploaded pool.

- **Requirements:**
    - **Similarity Scoring:** Calculate a "Compatibility Score" (0–100%) based on how the candidate's semantic profile aligns with the JD requirements.
    - **Weighted Ranking:** Prioritize candidates based on the depth of experience (e.g., 5 years of experience should rank higher than a mention of a skill in a single project).
    - **AI Justification:** For the top 5 candidates, generate a 2-sentence "Summary of Fit" explaining exactly why they were ranked highly (e.g., "Matches the 3+ years of React experience required and has a strong background in API design").

---

4. Minimum End-to-End Expectations

- **Recruiter Dashboard:** A landing page displaying active job roles, the number of resumes uploaded for each, and a "Top Talent" preview.
- **Upload & Parsing View:** A clean interface for bulk-uploading resumes with a real-time status indicating successful text extraction and parsing.
- **Ranking View:** A table showing candidates ranked by score, with filterable columns for "Years of Experience," "Top Skills," and the "AI Justification" snippet.

Submission Instructions

1. Push the completed project to a **GitHub repository**.
2. Ensure the repository includes:
    - Complete source code
    - `README.md` with setup instructions
    - `.env.example` (without sensitive keys)
    - API documentation (if applicable)
3. Share the **GitHub repository URL** as part of your submission.
4. **After completing the project, take screenshots of the application's output** (such as Login, Dashboard, CRUD operations, and GenAI features) and **email them to binu.christin@faithinfotech.in**.
5. The email should include:
    - Subject: **Python Full Stack with GenAI Machine Test Submission – <Your Name>**
    - Your Name
    - GitHub Repository Link
    - Attached screenshots of the application output